

COS 214 Practical 4

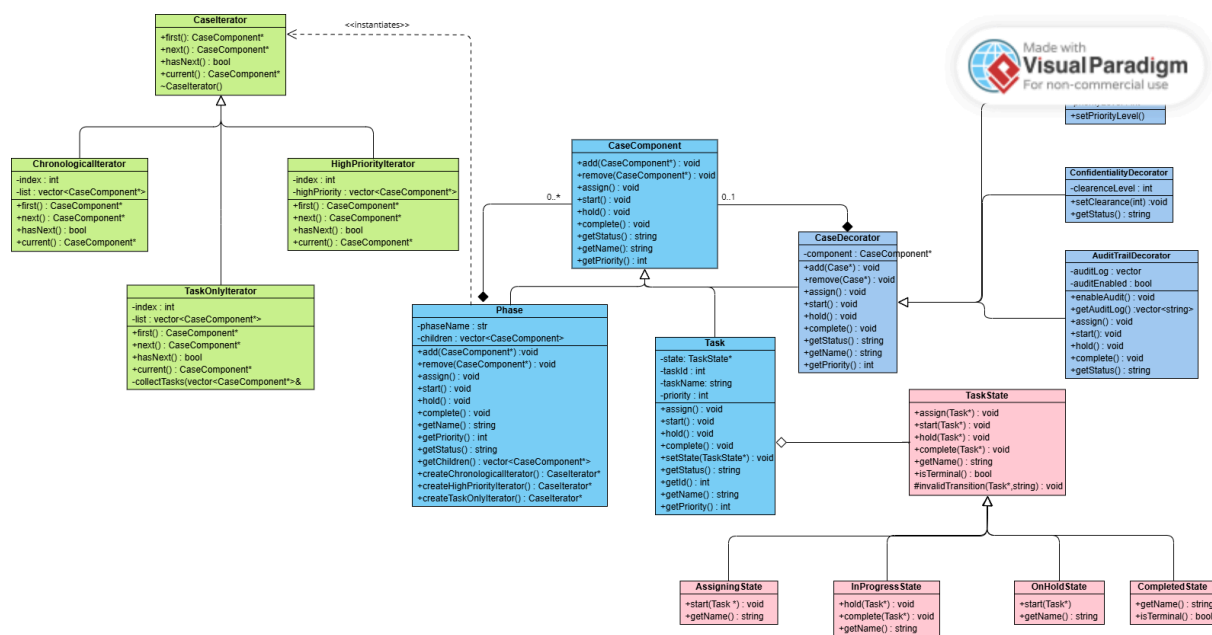
Thandolwethu Jantjies (u25114469)

Tumisho Matsiu (u25085892)

Yash Ghela (u25161360)

Task 1:

UML Diagram:



Description:

We chose a case management system. It organizes work into Phases (containers) and Tasks (individual work items) that progress through a strict workflow lifecycle. The system provides multiple views of the investigation (chronological, task-only, and high-priority) and allows dynamic addition of audit trails and confidentiality controls to sensitive tasks. This gives investigation teams complete visibility, process compliance, and audit readiness.

GoF participants

Composite Pattern:

- Component: CaseComponent
- Leaf: Task
- Composite: Phase

State Pattern:

- Context: Task

- State: TaskState
- ConcreteState: AssigningState, InProgressState, OnHoldState, CompletedState

Decorator Pattern:

- Component: CaseComponent
- ConcreteComponent: Task
- Decorator: CaseDecorator
- ConcreteDecorator: AuditTrailDecorator, ConfidentialityDecorator, HighPriorityDecorator

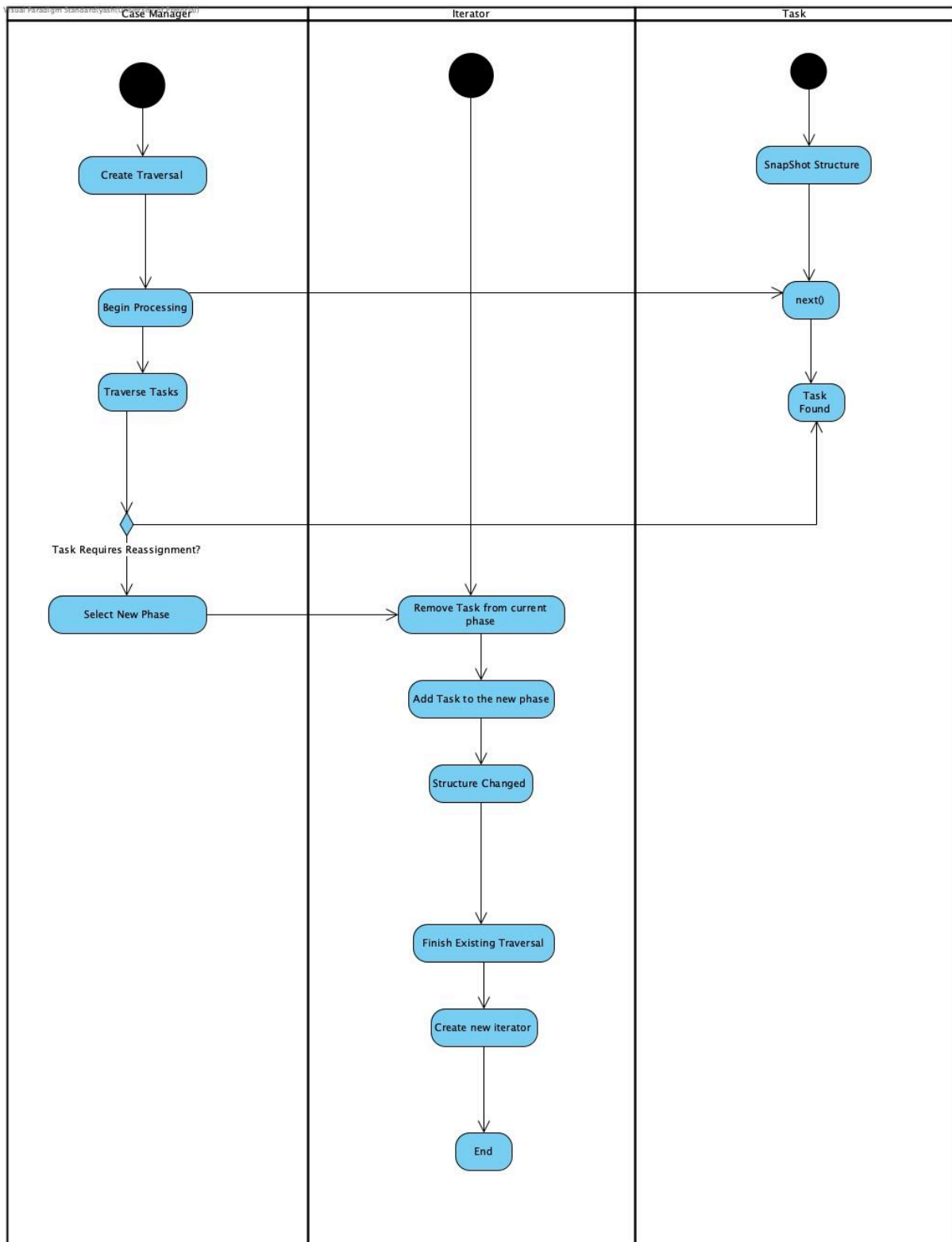
Iterator Pattern:

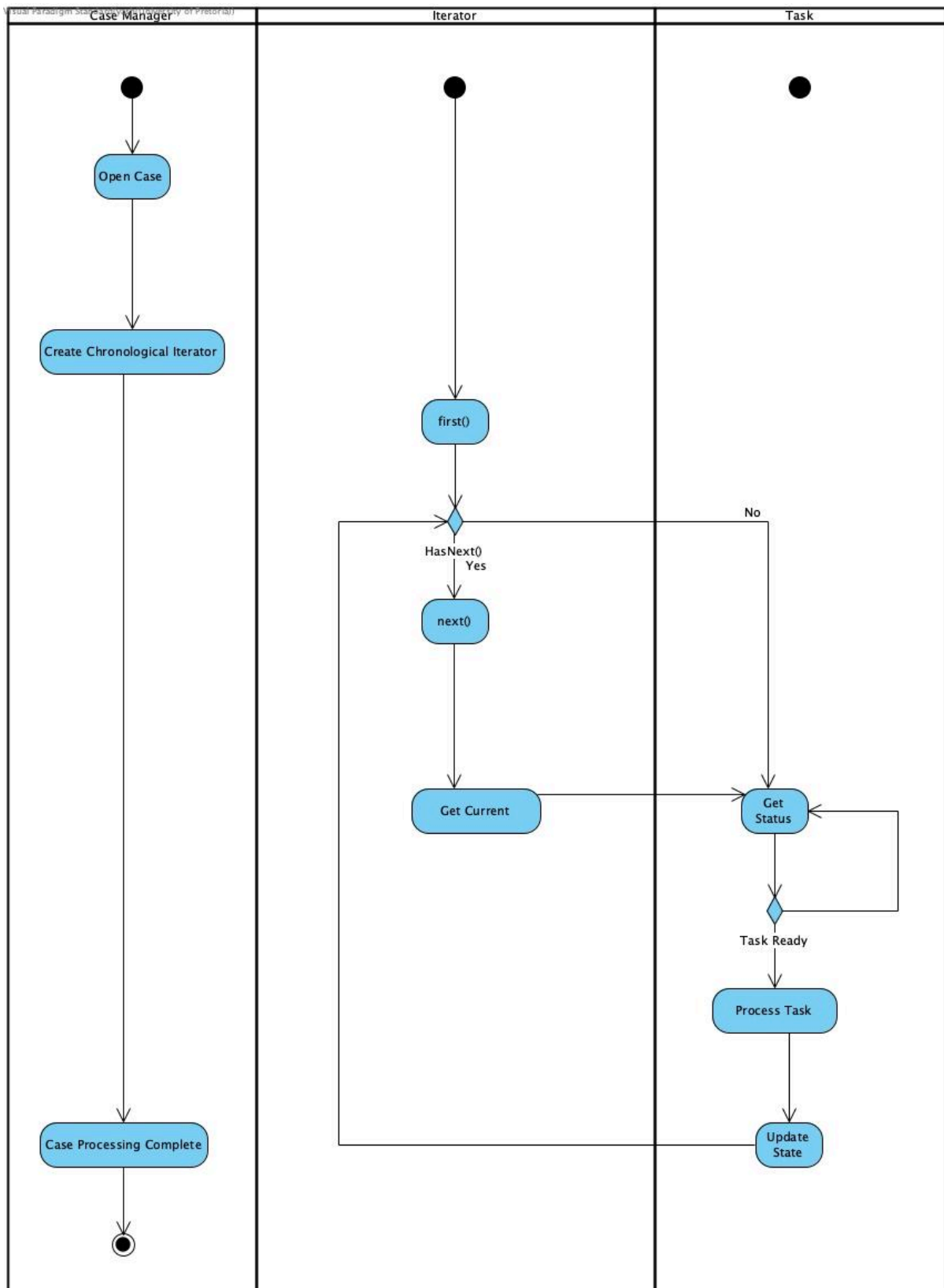
- Aggregate: Phase
- Iterator: CaseIterator
- ConcreteIterators : ChronologicalIterator, TaskOnlyIterator, HighPriorityIterator

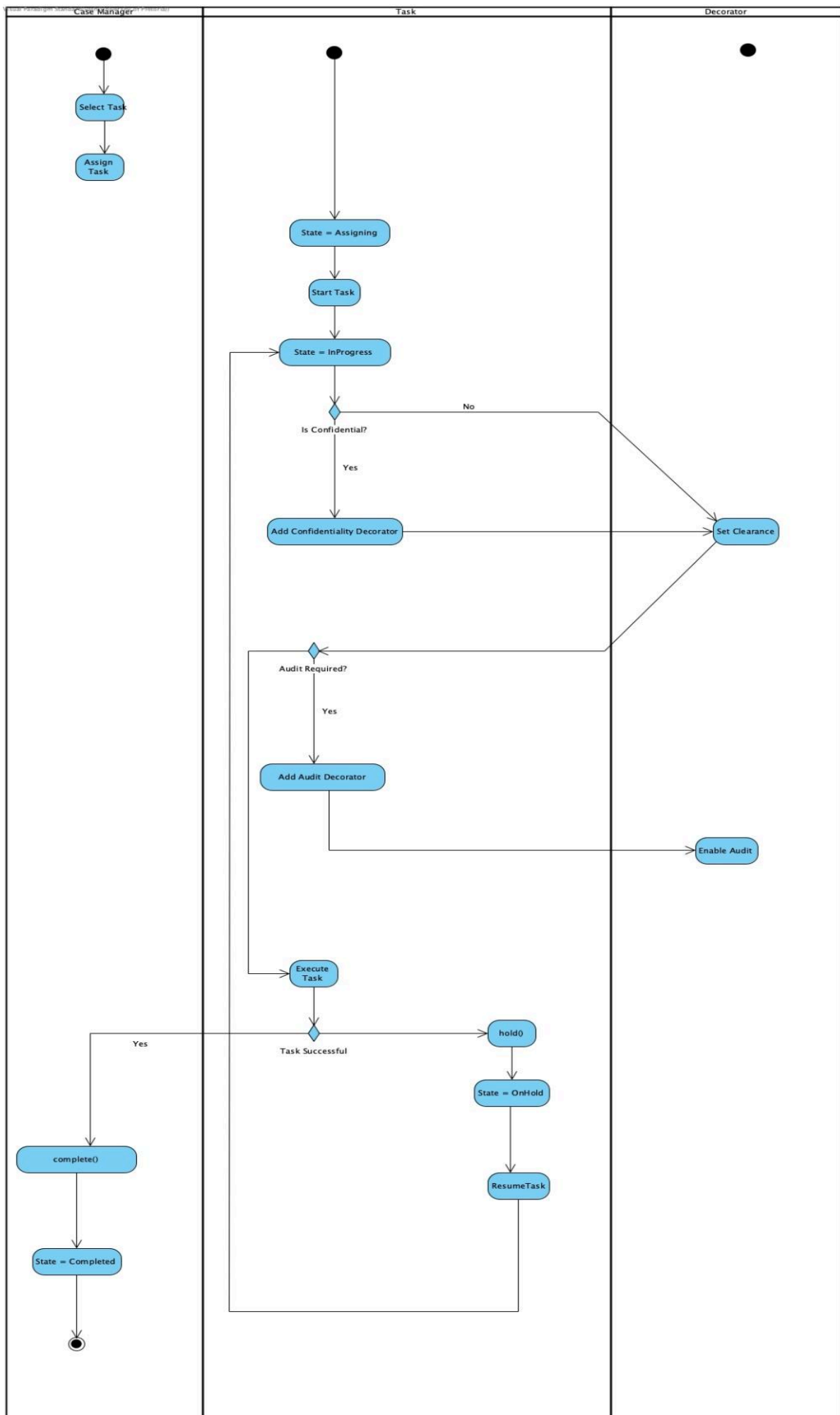
Ownership Rationale:

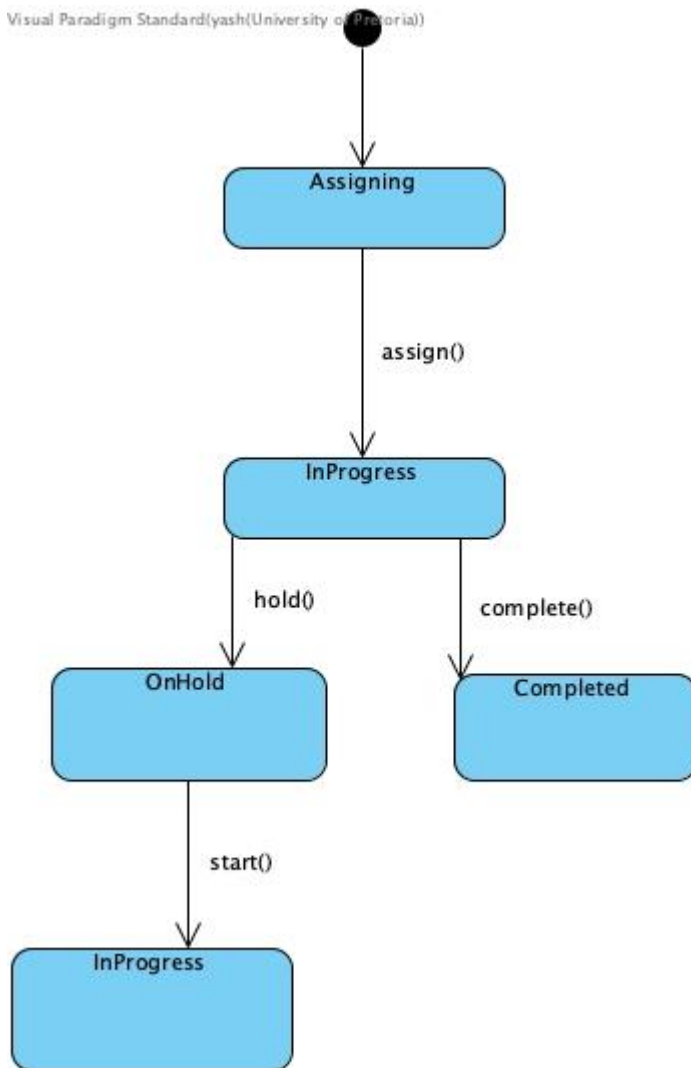
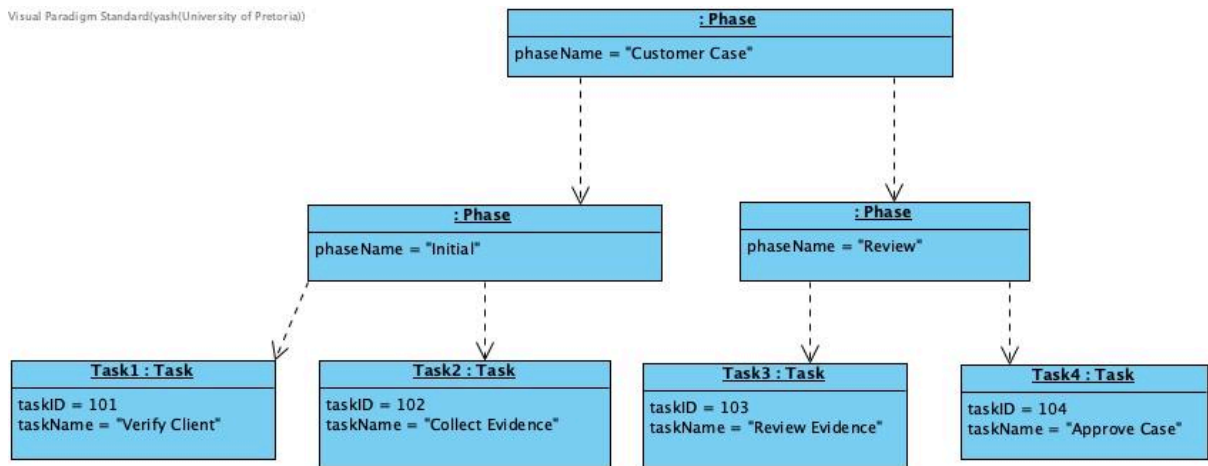
Phase owns its children (destructor deletes each one recursively, giving clean teardown of arbitrarily deep nesting), CaseDecorator owns the component it wraps (destructor deletes it, so deleting the outermost decorator cascades through the whole stack down to the underlying Task), and Task owns its current TaskState (deleted and replaced on every transition via setState). Iterators never own anything: they hold copies of pointers (a snapshot) for traversal, and are themselves owned and destroyed by whichever client created them.

Task 4:









Task5:

GDB Evidence:

Our runtimeScenarios function has caused a segmentation fault. The purpose of this function was to test different scenarios of ownership within our case system and the various state changes of tasks and how they behave.

```
RunTimeScenarios.cpp
48 CaseIterator* iterator = project->createChronologicalIterator();
49
50 while(iterator->hasNext()){
51     CaseComponent* comp = iterator->next();
52     std::cout<< comp->getStatus() <<std::endl;
53 }
54
55 //attempt run time changes
56
57 api->assign();
58 api->start();
59
60 api->setState(new InProgressState());
61
62 api->complete();
63
64 CaseComponent* decoratedapi = new AuditTrailDecorator(api, false);
65
66 //stacking decs
67
68 decoratedapi = new ConfidentialityDecorator(2,decoratedapi);
69

multi-thre Thread 0x7ffff7e9a7 In: RunTimeScenarios L52 PC: 0x55555559ffc
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00005555555559ffc in RunTimeScenarios () at RunTimeScenarios.cpp:52
(gdb) |
```

Breakpoint usage:

We can assess that comp returns a nullptr, as the next function passes through its last element and returns a nullptr to getStatus causing the segfault to occur. So on the final iteration hasNext() is true before the call to next(). next() increments the index past the last element returning a nullptr and getStatus() ends up dereferencing that nullptr causing a segfault

```
RunTimeScenarios.cpp
37 development->add(backend);
38
39 project->add(planning);
40 project->add(development);
41 project->add(testing);
42
43
44 //scenario 1 normal processing
45
46 std::cout<<"SCENARIO 1"<<std::endl;
47
48 CaseIterator* iterator = project->createChronologicalIterator();
49
50 while(iterator->hasNext()){
51     CaseComponent* comp = iterator->next();
52     std::cout<< comp->getStatus() <<std::endl;
53 }
54
55 //attempt run time changes
56
57 api->assign();
58 api->start();

B+>

multi-thre Thread 0x7ffff7e9a7 (src) In: RunTimeScenarios L52 PC: 0x55555559ffc5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, RunTimeScenarios () at RunTimeScenarios.cpp:52
(gdb) |
```

The nullptr is after line 52:

```
Breakpoint 1, RunTimeScenarios () at RunTimeScenarios.cpp:52
(gdb) print comp
$5 = (CaseComponent *) 0x0
```

The fix: change in traversals

replace `iterator->next()` with `iterator->current()` returns the current index allowing `getStatus` to return the current status of the component within the list of the `CaseComponents` and by having `iterator->next()` after the `getStatus()` function call prevents `nullptr` from being returned. Finally ensuring to delete the iterator to ensure memory safety.

Debugging result post fix (Valgrind):

```
==57292==
==57292== HEAP SUMMARY:
==57292==    in use at exit: 0 bytes in 0 blocks
==57292==   total heap usage: 156 allocs, 156 frees, 79,187 bytes allocated
==57292==
==57292== All heap blocks were freed -- no leaks are possible
==57292==
==57292== For lists of detected and suppressed errors, rerun with: -s
==57292== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

TASK 6:

Teamwork Reflection:

The tasks were equally distributed amongst team members in regards to their availability and skillset. Thando had a focus on setting up or foundational design with Yash adding onto this systems design. There was a constant feedback loop between team members in regards to implementation and overall design structure. Tumisho had a heavy basis on code testing and deployment. A great defining commit would be Thando's initial commit of the base implementation of the system, allowing Tumisho and Yash to better grasp the implementation of the design patterns and how the different components of the Case system relate and interact with each other. Yash's runtime scenarios commit enabled us to see the system in use before an official demoing main.